

Universidade de Brasília
Faculdade Gama
Engenharia de Software



UnB

Disciplina: Testes de Software

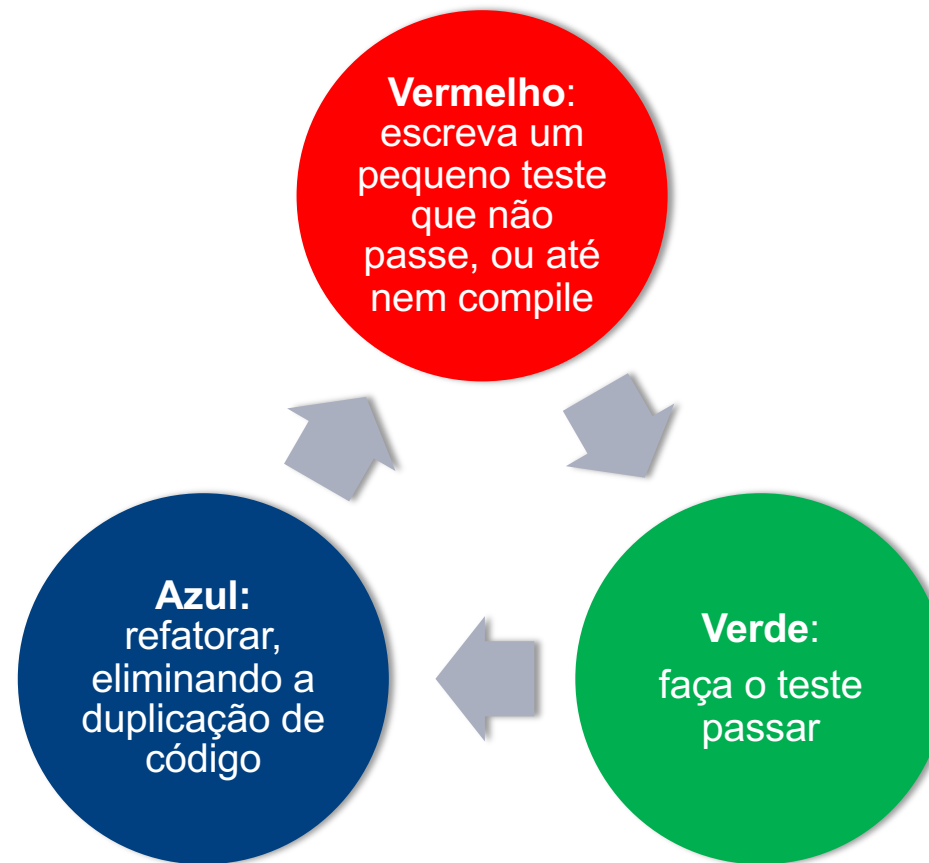
TDD

Elaine Venson

elainevenson@unb.br

Mantra do TDD

Vermelho -> Verde -> Refatorar

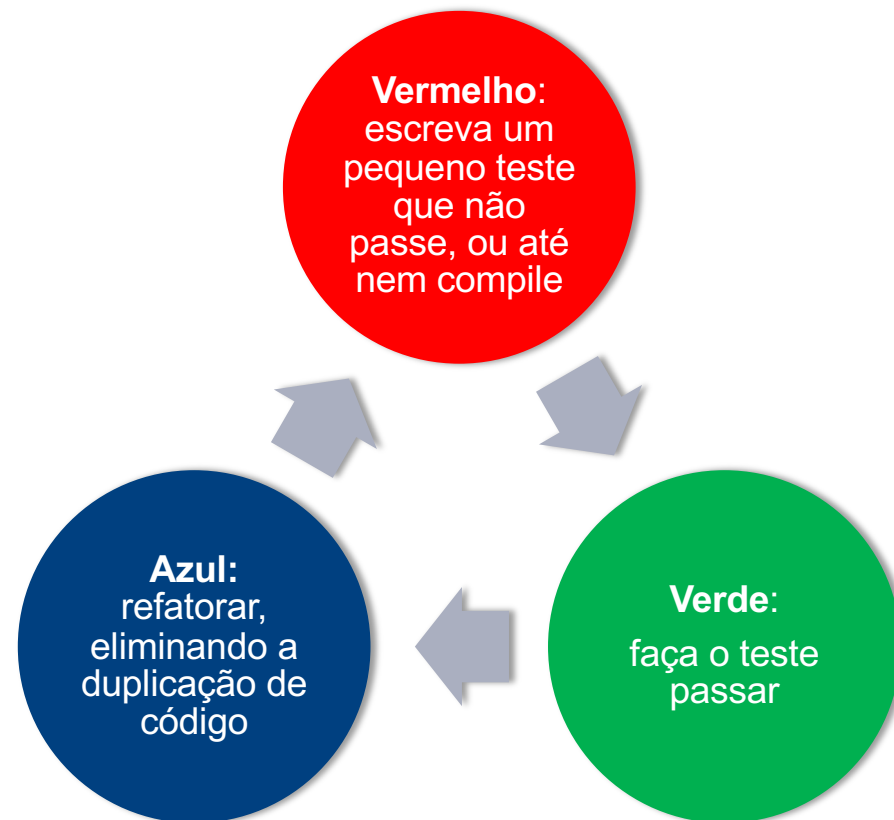


Passos do TDD

CICLO:

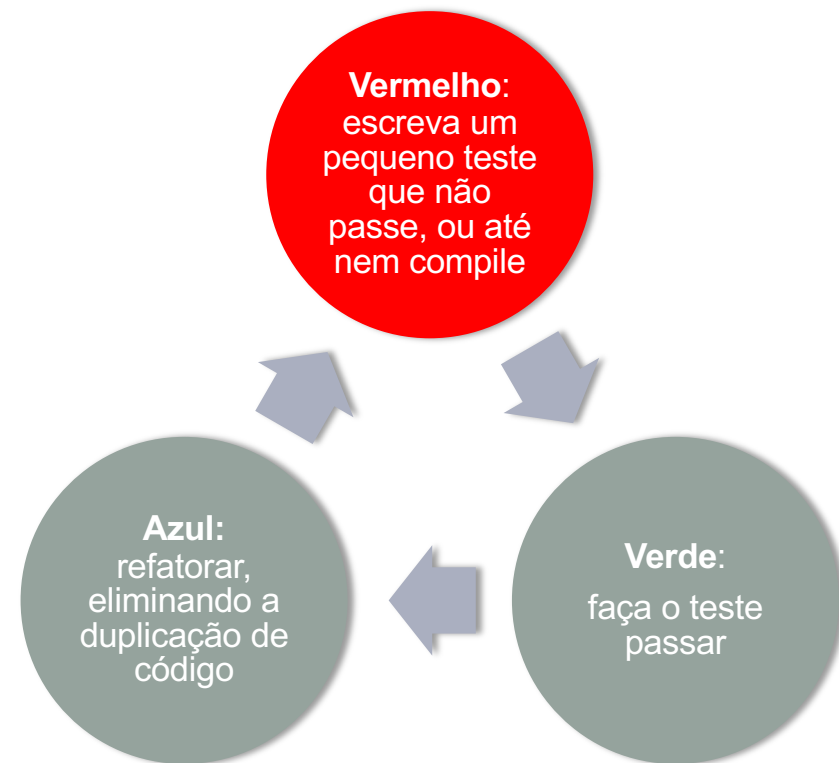
Vermelho -> Verde -> Refatorar

1. Escreva um teste
2. Execute todos os testes
3. Escreva o código de implementação
4. Execute todos os testes
5. Refatore
6. Execute todos os testes



Escrevendo os Testes: Vermelho

- O teste deve falhar!
- Se não falhar, algo está errado:
 - Descreve algo que já existe (já está implementado)
 - Ou foi escrito de forma incorreta (falso positivo)
- Usar nomes de método que descrevam o teste:
 - Ajuda a descobrir por que alguns testes falharam ou quando a cobertura deve ser aumentada com mais testes



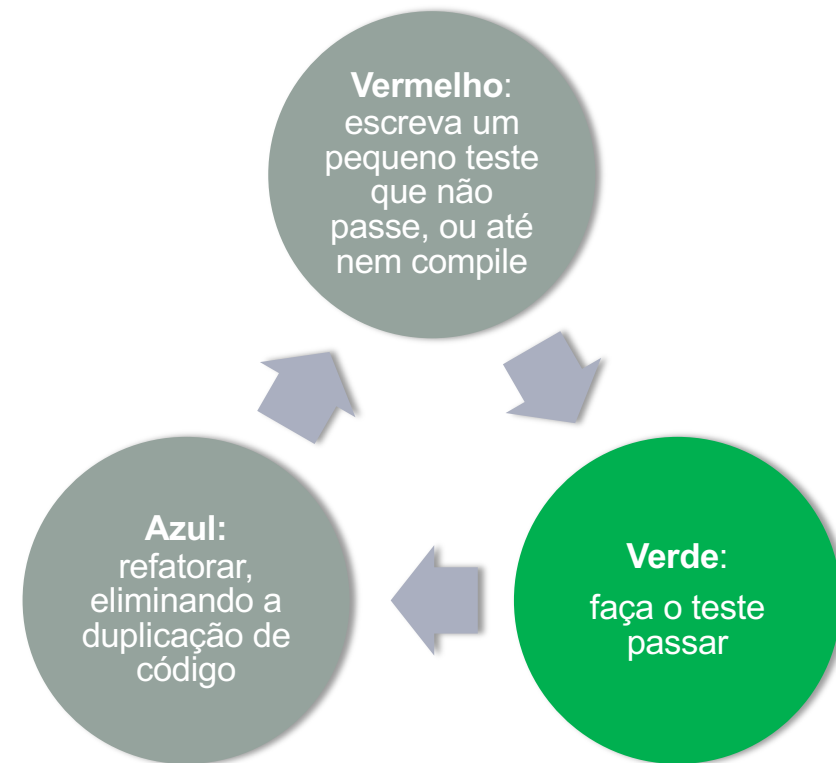
Escrevendo o código: Verde

- Ao terminar a implementação para atender último teste criado, todos os testes devem passar
- Se falhar:
 - O teste pode estar errado, ou
 - A implementação está errada



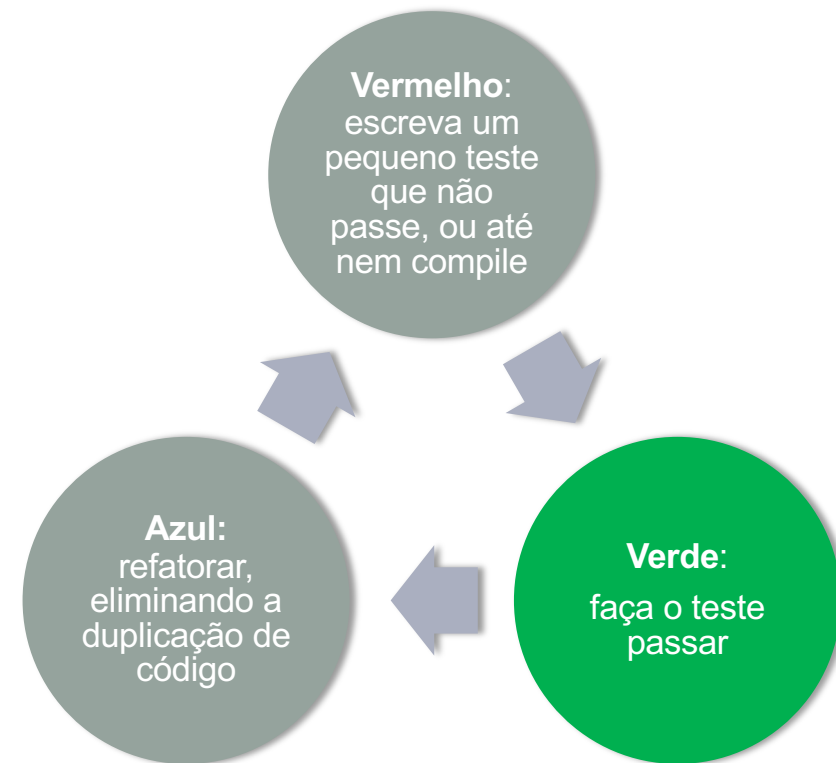
Escrevendo o código: Verde

- Se algum outro teste, **que não o último**, falhar:
 - O código foi quebrado!
 - Se a identificação do problema não for rápida, o código deve ser revertido.



Escrevendo o código: Verde

- Escrever código **apenas o suficiente** para o teste passar
- A implementação deve ser rápida!
- Importante rodar TODOS os testes para garantir que o novo código não quebra um teste existente



Refatorando: Azul

- Melhoria da qualidade do código com a proteção dos testes
- Não deve alterar a funcionalidade existente
- Se algum teste falhar, a refatoração quebrou o código:
 - As mudanças no código devem ser revertidas
- Não ocorre necessariamente em todos os ciclos



TDD: Velocidade

- As três fases são repetidas continuamente em **ciclos curtos**.
- É como um jogo de ping-pong muito rápido! Quanto mais profissional, mais rápido é o jogo.
- Veteranos de TDD tendem a não gastar mais de um minuto em cada lado da mesa (teste e implementação).
- Ciclos demorados indicam que o escopo do teste está grande e deve ser quebrado em testes menores.



TDD não é teste

- TDD é uma forma de abordar o design
- É uma maneira de focar nos requisitos e na implementação de apenas uma coisa por vez - organizar os pensamentos e estruturar melhor o código
- Ser capaz de reorganizar o código com a confiança de que nenhuma funcionalidade foi quebrada é um grande impulso para a qualidade
- O principal objetivo do TDD é o projeto de código testável -> os testes são um subproduto útil do TDD

Exemplo: Conversor de Números Romanos

Requisito

- Implementar um programa que:
- Recebe um número romano (String)
- Retorna o valor decimal correspondente

Exemplos

- $I \rightarrow 1$
- $V \rightarrow 5$
- $XV \rightarrow 15$
- $XXIV \rightarrow 24$

Exemplos de Casos de Teste

Casos Simples

- "I" $\rightarrow$ 1
- "V" $\rightarrow$ 5
- "X" $\rightarrow$ 10

Múltiplos caracteres

- "II" $\rightarrow$ 2
- "III" $\rightarrow$ 3
- "XVII" $\rightarrow$ 17

Notação subtrativa

- "IV" $\rightarrow$ 4
- "IX" $\rightarrow$ 9
- "XIV" $\rightarrow$ 14

Primeiro Passo do TDD

Escolher o caso mais simples

Primeiro exemplo

- "I" deve retornar 1

Estratégia

- Escrever o teste primeiro
- Mesmo que o código ainda não exista

Primeiro Teste - RED

```
@Test
void shouldUnderstandSymbolI() {
    RomanNumeralConverter roman =
        new RomanNumeralConverter();

    int number = roman.convert("I");

    assertThat(number).isEqualTo(1);
}
```

Situação

✗ O código não compila
A classe ainda não existe

Primeira Implementação - GREEN

Fazer o teste compilar

```
public class RomanNumeralConverter {  
  
    public int convert(String numberInRoman) {  
        return 0;  
    }  
}
```

Resultado

- O teste compila
- O teste falha

Fazer o Teste Passar

Implementação mínima

```
public int convert(String numberInRoman) {  
    return 1;  
}
```

Resultado

-  O teste passa

Observação

- A solução ainda não é geral.
- Mas isso é aceitável no TDD.

Novo Ciclo – RED

Novo Teste

- Novo requisito: "V" deve retornar 5

```
@Test
void shouldUnderstandSymbolV() {

    RomanNumeralConverter roman =
        new RomanNumeralConverter();

    int number = roman.convert("V");

    assertThat(number).isEqualTo(5);
}
```

Fazer o Teste Passar - GREEN

```
public int convert(String numberInRoman) {  
  
    if(numberInRoman.equals("I"))  
        return 1;  
  
    if(numberInRoman.equals("V"))  
        return 5;  
  
    return 0;  
}
```

Resultado

✓ Ambos os testes passam

Refatoração

Melhorando o Código

Problema

- Muitos ifs deixam o código difícil de manter.

Solução

- Usar uma tabela de conversão.

Refatoração

Implementação Simplificada

```
private static Map<String, Integer> table =  
    new HashMap<>() {{  
  
        put("I", 1);  
        put("V", 5);  
        put("X", 10);  
        put("L", 50);  
        put("C", 100);  
        put("D", 500);  
        put("M", 1000);  
  
    }};
```

```
public int convert(String numberInRoman) {  
    return table.get(numberInRoman);  
}
```

Resultado

- ✓ Código mais limpo
- ✓ Mais extensível

Novo Ciclo – Múltiplos Caracteres - RED

Requisito

- Números com Múltiplos Caracteres

Exemplos

- "II" → 2
- "XVIII" → 18

```
@Test
void shouldUnderstandMultipleCharNumbers() {

    RomanNumeralConverter roman =
        new RomanNumeralConverter();

    int converted =
        roman.convert("II");

    assertThat(converted)
        .isEqualTo(2);
}
```

Fazer o Teste Passar - GREEN

Implementação

```
public int convert(String numberInRoman) {  
  
    int finalNumber = 0;  
  
    for(int i = 0;  
        i < numberInRoman.length();  
        i++) {  
  
        finalNumber +=  
            table.get(  
                numberInRoman.charAt(i)  
            );  
    }  
  
    return finalNumber;  
}
```

Resultado

- Casos simples funcionando

Novo Ciclo – Notação Subtrativa - RED

Novo Desafio

Exemplos

- $IV \rightarrow 4$
- $IX \rightarrow 9$
- $XL \rightarrow 40$

Regra

- Se um símbolo menor vier antes de um maior:
→ Subtrair ao invés de somar

Fazer o Teste Passar - GREEN

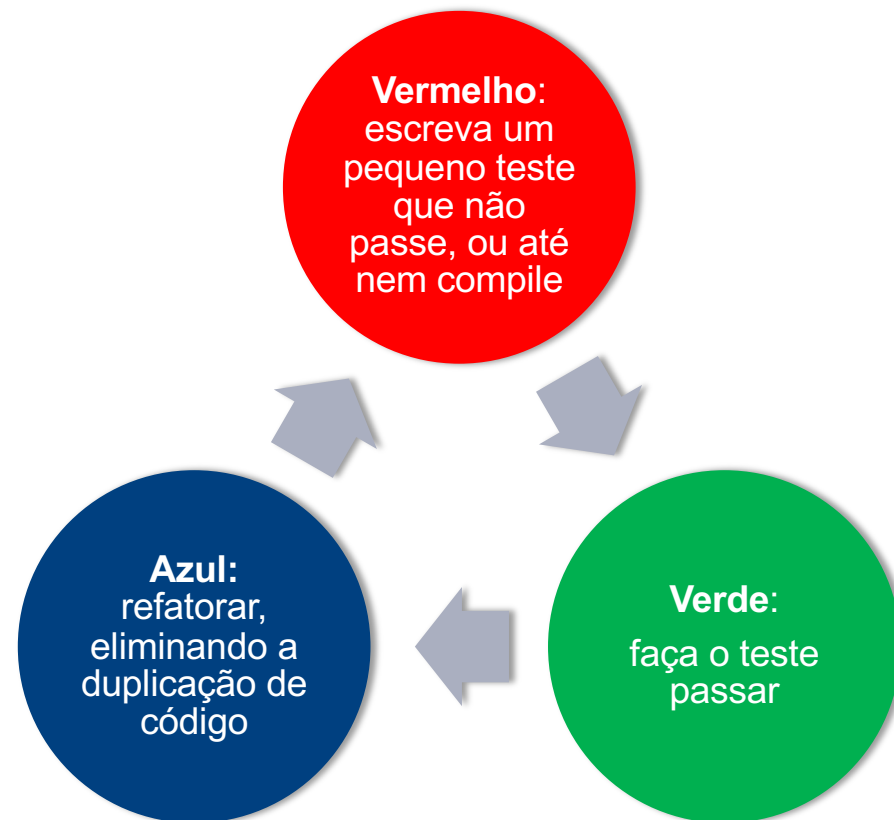
```
public int convert(String numberInRoman) {  
  
    int finalNumber = 0;  
    int lastNeighbor = 0;  
  
    for(int i = numberInRoman.length()-1;  
        i >= 0;  
        i--) {  
  
        int current =  
            table.get(numberInRoman.charAt(i));  
  
        int multiplier = 1;  
  
        if(current < lastNeighbor)  
            multiplier = -1;  
  
        finalNumber += current * multiplier;  
  
        lastNeighbor = current;  
    }  
  
    return finalNumber;  
}
```


Passos do TDD

CICLO:

Vermelho -> Verde -> Refatorar

1. Escreva um teste
2. Execute todos os testes
3. Escreva o código de implementação
4. Execute todos os testes
5. Refatore
6. Execute todos os testes



TDD vale a pena?

Parte 1 - Leitura Dirigida

Leitura livro ANICHE (15 min)

Grupo	Tema
Grupo 1 e 6	8.3.1 — To TDD or not to TDD
Grupo 2 e 7	8.3.2 — TDD 100% of the time?
Grupo 3 e 8	8.3.4 — What does research say about TDD?
Grupo 4 e 9	8.3.5 — Other schools of TDD
Grupo 5 e 10	8.3.6 — TDD and proper testing

Parte 2 — Produção do grupo

Cada grupo deve responder: (20 min)

1. Qual é a principal ideia da seção?
2. Qual argumento do autor vocês concordam mais?
3. Qual argumento vocês contestariam?
4. Em que contexto TDD parece útil?
5. Em que contexto TDD parece pouco útil?

Parte 3 - Debate

Produtividade

- TDD realmente reduz velocidade?
- O ganho posterior compensa?

Qualidade

- TDD melhora design?
- Ou apenas força escrita de testes?

Pesquisa científica

- Por que os estudos têm resultados contraditórios?
- Os experimentos acadêmicos representam a indústria?

Parte 3 - Debate

Prática profissional

- TDD funciona em startups?
- Funciona em sistemas legados?
- Funciona em governo?
- Funciona em IA?
- Funciona em times juniores?

IA Generativa

- “Equipe criando prompts e pipelines de LLM.”

TDD funciona nesse contexto?

Referência

- Aniche, M., Effective Software Testing: A developer's Guide. Manning, 2021.
- Farcic, V., & Garcia, A. (2018). Test-driven java development, second edition : Invoke tdd principles for end-to-end application development, 2nd edition. Packt Publishing, Limited. Created from univbrasil-ebooks on 2022-03-25 13:59:55.